

60 SQL Interview Questions with Solutions (Data Engineering Focus)

Assumed Table: employees

EMP_ID	NAME	DEPARTMENT	SALARY	MANAGER_ID	DOB
101	Amit	IT	75000	NULL	1998-04-12
102	Sneha	HR	50000	101	1997-08-21
103	Rahul	Finance	65000	101	1995-02-15
104	Simran	IT	85000	101	1996-10-30
105	Vikram	Sales	45000	103	1999-07-18
106	Saurabh	HR	55000	102	1998-12-01
107	Karan	Finance	70000	103	1994-05-11
108	Anjali	Sales	48000	105	2000-03-25
109	Sonia	IT	85000	104	1997-09-09
110	Priya	IT	NULL	104	1998-01-20
111	Rahul	Finance	65000	101	1995-02-15

Assumed Table: orders

ORDER_ID	EMP_ID	ORDER_AMOUNT
1001	101	5000
1002	102	7000
1003	104	12000
1004	107	9000
1005	109	15000

BASIC SQL

1. Select name and salary from employees table.

```
SELECT name, salary  
FROM employees;
```

2. Select all columns from employees table.

```
SELECT *  
FROM employees;
```

3. Retrieve employees with salary greater than 50000.

```
SELECT *  
FROM employees  
WHERE salary > 50000;
```

4. Find all distinct departments in employees table.

```
SELECT DISTINCT department  
FROM employees;
```

5. Sort employees by salary in descending order.

```
SELECT *  
FROM employees  
ORDER BY salary DESC;
```

6. Get top 3 rows from employees table.

```
SELECT *  
FROM employees  
LIMIT 3;
```

7. Count total number of employees.

```
SELECT COUNT(*) AS total_employees  
FROM employees;
```

8. Find total sum of all salaries.

```
SELECT SUM(salary) AS total_salary  
FROM employees;
```

9. Find average salary of employees.

```
SELECT AVG(salary) AS avg_salary  
FROM employees;
```

10. Find maximum salary in employees table.

```
SELECT MAX(salary) AS max_salary  
FROM employees;
```

11. Find minimum salary in employees table.

```
SELECT MIN(salary) AS min_salary  
FROM employees;
```

GROUP BY / HAVING

12. Get count of employees in each department.

```
SELECT department,  
       COUNT(*) AS employee_count  
FROM employees  
GROUP BY department;
```

13. Find departments where average salary is greater than 50000.

```
SELECT department,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department  
HAVING AVG(salary) > 50000;
```

FILTERING

14. Find employees whose name starts with 'S'.

```
SELECT *  
FROM employees  
WHERE name LIKE 'S%';
```

15. Find employees belonging to IT or HR departments.

```
SELECT *  
FROM employees  
WHERE department IN ('IT', 'HR');
```

16. Find employees with salary between 40000 and 60000.

```
SELECT *  
FROM employees  
WHERE salary BETWEEN 40000 AND 60000;
```

17. Find employees with NULL manager_id.

```
SELECT *  
FROM employees  
WHERE manager_id IS NULL;
```

18. Categorize employees as HIGH or LOW salary using CASE.

```
SELECT name,  
       salary,  
       CASE  
         WHEN salary >= 50000 THEN 'HIGH'  
         ELSE 'LOW'  
       END AS salary_category  
FROM employees;
```

JOINS

19. INNER JOIN

```
SELECT *  
FROM employees e  
INNER JOIN orders o  
ON e.emp_id = o.emp_id;
```

20. LEFT JOIN

```
SELECT *  
FROM employees e  
LEFT JOIN orders o  
ON e.emp_id = o.emp_id;
```

21. RIGHT JOIN

```
SELECT *  
FROM employees e  
RIGHT JOIN orders o  
ON e.emp_id = o.emp_id;
```

22. FULL OUTER JOIN

```
SELECT *  
FROM employees e
```

```
FULL OUTER JOIN orders o
ON e.emp_id = o.emp_id;
```

23. CROSS JOIN

```
SELECT *
FROM employees e
CROSS JOIN orders o;
```

24. SELF JOIN to find manager-employee relation.

```
SELECT e.name AS employee_name,
       m.name AS manager_name
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;
```

UNION

25. UNION

```
SELECT department
FROM employees
UNION
SELECT department
FROM employees;
```

26. UNION ALL

```
SELECT department
FROM employees
UNION ALL
SELECT department
FROM employees;
```

SUBQUERY / EXISTS / ANY / ALL

27. Query using subquery to filter based on aggregated value.

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

28. Use EXISTS to check related records.

```
SELECT *
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE e.emp_id = o.emp_id
);
```

29. Use ANY to compare salary with HR salaries.

```
SELECT *
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE department = 'HR'
);
```

30. Use ALL to compare salary with all HR salaries.

```
SELECT *
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE department = 'HR'
);
```

DDL / DML

31. Create student table.

```
CREATE TABLE student (  
    student_id INT,  
    student_name STRING,  
    marks INT  
);
```

32. Insert record into student table.

```
INSERT INTO student  
VALUES (1, 'Aman', 90);
```

33. Update record in student table.

```
UPDATE student  
SET marks = 95  
WHERE student_id = 1;
```

34. Delete record from student table.

```
DELETE FROM student  
WHERE student_id = 1;
```

35. Define PRIMARY KEY.

```
CREATE TABLE employee_pk (  
    emp_id INT PRIMARY KEY,  
    name STRING  
);
```


36. Define FOREIGN KEY.

```
CREATE TABLE employee_fk (  
    emp_id INT,  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
    REFERENCES departments(dept_id)  
);
```

VIEW / INDEX

37. Create VIEW for high salary employees.

```
CREATE VIEW high_salary_employees AS  
SELECT *  
FROM employees  
WHERE salary > 60000;
```

38. Create INDEX.

Note: Snowflake does not support traditional indexes.

Traditional SQL example:

```
CREATE INDEX idx_salary  
ON employees(salary);
```

WINDOW FUNCTIONS

39. Assign row numbers based on salary.

```
SELECT name,  
       salary,  
       ROW_NUMBER() OVER(ORDER BY salary DESC) AS row_num  
FROM employees;
```

40. Rank employees using RANK().

```
SELECT name,  
       salary,  
       RANK() OVER(ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

41. Rank employees using DENSE_RANK().

```
SELECT name,  
       salary,  
       DENSE_RANK() OVER(ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

42. PARTITION BY with ranking by department.

```
SELECT name,  
       department,  
       salary,  
       RANK() OVER(  
           PARTITION BY department  
           ORDER BY salary DESC  
       ) AS dept_rank  
FROM employees;
```

43. LEAD function.

```
SELECT name,  
       salary,  
       LEAD(salary) OVER(ORDER BY salary) AS next_salary  
FROM employees;
```

44. LAG function.

```
SELECT name,  
       salary,  
       LAG(salary) OVER(ORDER BY salary) AS previous_salary  
FROM employees;
```

CTE

45. Create CTE for high earning employees.

```
WITH high_earners AS (  
    SELECT *  
    FROM employees  
    WHERE salary > 50000  
)  
SELECT *  
FROM high_earners;
```

ADVANCED SQL

46. Find second highest salary.

```
SELECT MAX(salary) AS second_highest_salary  
FROM employees  
WHERE salary < (  
    SELECT MAX(salary)  
    FROM employees  
);
```

Alternative:

```
SELECT salary  
FROM (  
    SELECT salary,  
           DENSE_RANK() OVER(ORDER BY salary DESC) AS rnk  
    FROM employees  
) t  
WHERE rnk = 2;
```

47. Find Nth highest salary.

```
SELECT salary  
FROM (  
    SELECT salary,  
           DENSE_RANK() OVER(ORDER BY salary DESC) AS rnk
```

```
        FROM employees
    ) t
    WHERE rnk = N;
```

48. Find duplicate records.

```
SELECT name,
       department,
       salary,
       COUNT(*) AS duplicate_count
FROM employees
GROUP BY name, department, salary
HAVING COUNT(*) > 1;
```

49. Delete duplicate records keeping one entry.

```
DELETE FROM employees
WHERE emp_id IN (
    SELECT emp_id
    FROM (
        SELECT emp_id,
               ROW_NUMBER() OVER(
                   PARTITION BY name, department, salary
                   ORDER BY emp_id
               ) AS rn
        FROM employees
    ) t
    WHERE rn > 1
);
```

NULL HANDLING

50. Replace NULL values using COALESCE.

```
SELECT name,
       COALESCE(salary, 0) AS salary
FROM employees;
```

51. Use NULLIF.

```
SELECT NULLIF(100, 100);
```

STRING FUNCTIONS

52. Convert names to uppercase.

```
SELECT UPPER(name)
FROM employees;
```

53. Convert names to lowercase.

```
SELECT LOWER(name)
FROM employees;
```

54. Find length of employee names.

```
SELECT name,
       LENGTH(name) AS name_length
FROM employees;
```

DATE FUNCTIONS

55. Get current date.

```
SELECT CURRENT_DATE();
```

56. Get current date and time.

```
SELECT CURRENT_TIMESTAMP();
```

IMPORTANT INTERVIEW QUESTIONS

57. Get top 3 salaries.

```
SELECT *
FROM (
    SELECT name,
           salary,
           DENSE_RANK() OVER(ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk <= 3;
```

58. Calculate employee age.

```
SELECT name,
       dob,
       DATEDIFF(YEAR, dob, CURRENT_DATE()) AS age
FROM employees;
```

59. Find employees who have no orders.

```
SELECT e.*
FROM employees e
LEFT JOIN orders o
ON e.emp_id = o.emp_id
WHERE o.emp_id IS NULL;
```

60. Find highest paid employee in each department.

```
SELECT *
FROM (
    SELECT name,
           department,
           salary,
           RANK() OVER(
               PARTITION BY department
               ORDER BY salary DESC
           ) AS rnk
    FROM employees
)
```

```
) t  
WHERE rnk = 1;
```

MOST IMPORTANT QUESTIONS FOR DATA ENGINEERING INTERVIEWS

1. GROUP BY + HAVING
2. JOINS
3. WINDOW FUNCTIONS
4. CTE
5. SECOND/NTH HIGHEST SALARY
6. DUPLICATE RECORDS
7. TOP N PER GROUP
8. LEAD/LAG
9. EMPLOYEES WITH NO ORDERS
10. HIGHEST SALARY PER DEPARTMENT
11. DEDUPLICATION USING ROW_NUMBER()
12. NULL HANDLING
13. EXISTS vs IN
14. ANY vs ALL
15. PERFORMANCE + OPTIMIZATION QUESTIONS